

SOK-1305 Statistisk læring for økonomer

Kapittel 8.1: Tre-baserte metoder

Tema: Beslutningstrær, oppdeling av prediktorrommet og beskjæring

ISLP kapittel 8.1 — Regresjonstrær, klassifiseringstrær, pruning

Læringsmål

Etter denne forelesningen skal du kunne:

1. **Lese** et beslutningstre og regne ut en prediksjon for hånd.
2. **Oversette** mellom et tre og den tilhørende oppdelingen av prediktorrommet.
3. **Forklare** hva rekursiv binær splitting gjør, og hvorfor den er *greedy*.
4. **Regne** RSS for en kandidat-splitt og velge den beste.
5. **Begrunne** hvorfor vi beskjærer (*pruner*) treet, og hva α gjør i kostnadskompleksitet.
6. **Skille** Gini-indeks, entropi og klassifikasjonsfeil — og si når hver brukes.
7. **Trene** og beskjære et tre i Python med `scikit-learn`.

Agenda

1. Intuisjon: hvordan tenker et tre?
2. Regresjonstrær og **Hitters**-datasettet.
3. Terminologi: noder, greiner og blader.
4. Fra tre til regioner — og tilbake.
5. Hvordan bygges treet? Rekursiv binær splitting.
6. Overtilpasning og beskjæring (*pruning*).
7. Klassifiseringstrær: Gini-indeks og entropi.
8. Trær versus lineære modeller.
9. Fordeler, ulemper — og Python.

Merk: Plassering i kurset

Kapittel 3 og 4 ga oss modeller som var én ligning for hele datasettet. Trær gjør noe helt annet — de deler datasettet i biter og gir hver bit sin egen enkle prediksjon. Det er samme mål (estimere f i $Y = f(X) + \varepsilon$), men en helt annen strategi.

Motivasjon: du bruker allerede beslutningstrær

En banksaksbehandler vurderer et lånesøknad. Hun spør ikke “hva er $\hat{\beta}_1 \cdot \text{inntekt} + \hat{\beta}_2 \cdot \text{gjeld} + \dots$?”

Hun spør: *Er inntekten over 500 000? Hvis ja: er gjeldsgraden under 4? Hvis ja: innvilg.*

Eksempel: Sekvensielle ja/nei-spørsmål

Et beslutningstre er akkurat dette — men der spørsmålene og terskelverdiene er **estimert fra data** i stedet for bestemt av en regel i en håndbok.

Lineær regresjon

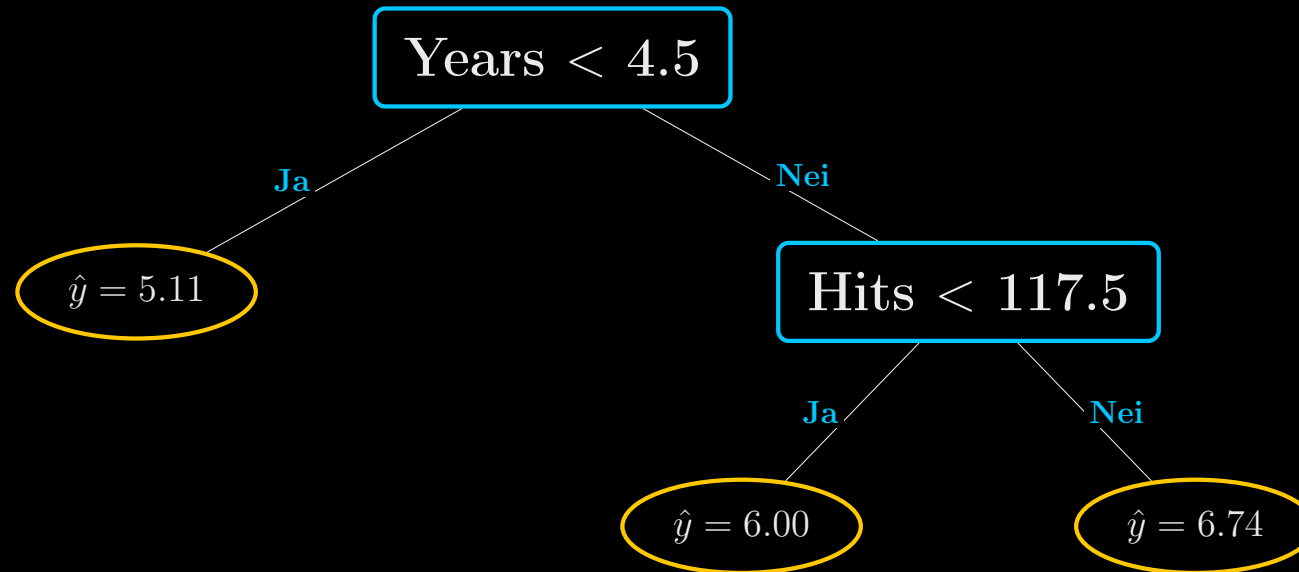
Én global ligning for alle
Effekten av X_j er den samme overalt
Interaksjoner må spesifiseres manuelt
Ekstrapolerer utenfor datasettet

Beslutningstre

Mange lokale gjennomsnitt
Effekten av X_j kan variere
Interaksjoner oppstår automatisk
Ekstrapolerer aldri

Eksempel: Lønn i Major League Baseball (Hitters)

Vi predikerer $\log(\text{Salary})$ ut fra **Years** (antall år i ligaen) og **Hits** (antall treff forrige sesong). $n = 263$ spillere. Vi logger lønna fordi fordelingen er høyreskjev.



Tallet i hvert blad er **gjennomsnittet av $\log(\text{Salary})$** for treningsobservasjonene som havner der. Tilbake til kroner: $1000 \cdot e^{5.107} = \$165\,174$, $1000 \cdot e^{5.999} = \$402\,834$, $1000 \cdot e^{6.740} = \$845\,346$.

Spørsmål: Les treet

Bruk treet på forrige side.

1. Hva predikerer treet for en spiller med **Years** = **2** og **Hits** = **200**?
2. Hva predikerer treet for en spiller med **Years** = **12** og **Hits** = **90**?
3. Hva predikerer treet for en spiller med **Years** = **12** og **Hits** = **91**?
4. To spillere har begge **Years** = 3. Den ene har 20 hits, den andre har 220. Hvor stor er forskjellen i predikert lønn?

Ta 3 minutter. Snakk med sidemannen. Ikke se på neste side.

Svar — og hva de avslører

1. Years = 2 < 4.5 $\Rightarrow$ venstre blad. $\hat{y} = 5.11$, dvs. ca. \$165 000. **Hits brukes ikke i det hele tatt.**
2. Years = 12 $\geq$ 4.5, Hits = 90 < 117.5 $\Rightarrow \hat{y} = 6.00$, ca. \$403 000.
3. Years = 12, Hits = 91 $\Rightarrow$ samme blad, samme svar: $\hat{y} = 6.00$.
4. **Null forskjell.** Begge er i venstre blad.

Merk: Tre viktige egenskaper faller ut av dette

- **Prediksjonen er trinnvis konstant.** Hits kan endre seg fra 90 til 117 uten at prediksjonen rører seg — og så hopper den 0.74 log-enheter ved 117.5.
- **Treet modellerer interaksjon uten at vi ber om det.** Hits har effekt *bare* for erfarne spillere. En lineær modell $\beta_1 \text{Years} + \beta_2 \text{Hits}$ kan ikke gjøre dette uten et eksplisitt interaksjonsledd.
- **Variabelseleksjon er innebygget.** I venstre gren finnes ikke Hits.

Samme modell, annen tegning: oppdeling av prediktorrommet

Treet deler X -rommet i tre rektangler:

$$R_1 = \{X \mid \text{Years} < 4.5\}$$

$$R_2 = \{X \mid \text{Years} \geq 4.5, \text{ Hits} < 117.5\}$$

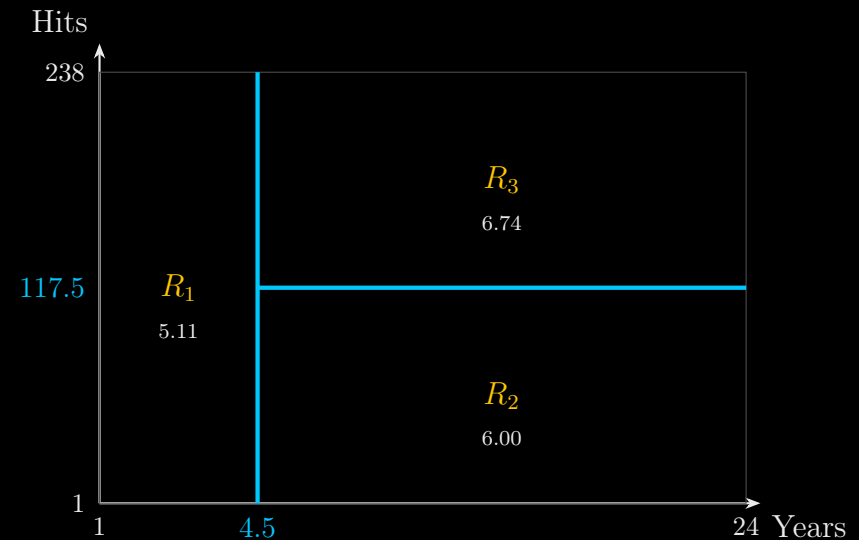
$$R_3 = \{X \mid \text{Years} \geq 4.5, \text{ Hits} \geq 117.5\}$$

Modellen kan skrives helt eksplisitt:

$$\hat{f}(X) = \sum_{m=1}^3 c_m \cdot \mathbf{1}(X \in R_m)$$

med $c_1 = 5.11$, $c_2 = 6.00$, $c_3 = 6.74$.

Treet og rektanglene er to bilder av nøyaktig samme funksjon.

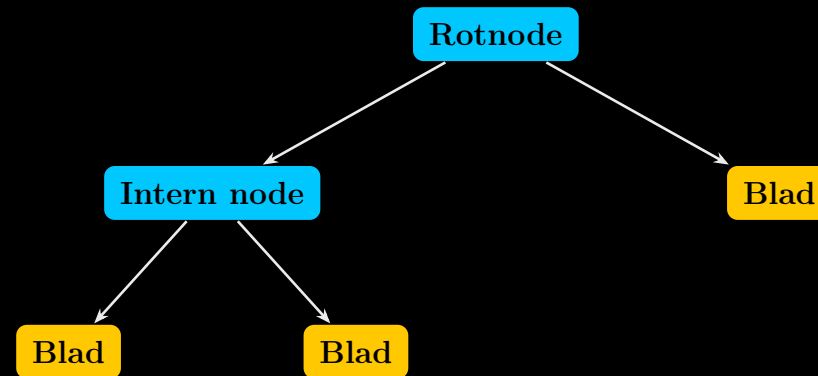


Merk: Hvorfor akkurat rektangler?

Regionene *kunne* hatt hvilken som helst form. Vi velger bokser fordi de er enkle å beskrive med ja/nei-spørsmål — og dermed enkle å tolke.

Anatomien til et tre

Ordene du trenger for å snakke presist om trær.



- **Intern node:** et sted hvor vi splitter på en variabel X_j ved en terskel s . Skriveregelen i ISLP: teksten $X_j < s$ beskriver **venstre** gren; høyre gren er $X_j \geq s$.
- **Grein (*branch*):** forbindelsen mellom noder.
- **Blad / terminalnode (*leaf*):** sluttpunktet. Her gis prediksjonen — gjennomsnittet (regresjon) eller flertallsklassen (klassifisering) blant treningsobservasjonene i bladet.
- $|T|$ betegner **antall blader**. Det er vårt mål på hvor komplekst treet er.

Trær tegnes opp-ned: rota øverst, bladene nederst.

Diskuter i grupper: Hva forteller treet oss egentlig?

Se på Hitters-treet igjen.

1. Kan vi si at **Years** er den “viktigste” variabelen? Hva betyr “viktig” her?
2. Treet sier at Hits ikke påvirker lønna for unge spillere. Er det en **sann sammenheng i verden**, eller en **egenskap ved algoritmen**?
3. En klubbdirektør sier: “Treet viser at hvis vi lar en spiller stå på benken i fem år, får han høyere lønn.” Hva er galt med den slutningen?
4. Er terskelen 4.5 år et **meningsfullt tall**? Hva ville skjedd med den hvis vi fjernet 20 tilfeldige spillere fra datasettet?

Diskuter

Hvordan bygger vi treet? To steg

Oppskriften

Steg 1. Del prediktorrommet — alle mulige verdier av $X_1, X_2, \dots, X_p$ — i J ikke-overlappende regioner $R_1, R_2, \dots, R_J$.

Steg 2. For enhver observasjon som faller i R_j , predikér det samme: gjennomsnittet av responsen for treningsobservasjonene i R_j .

Steg 2 er trivielt. All jobben ligger i steg 1. Målet er å finne bokser som minimiserer

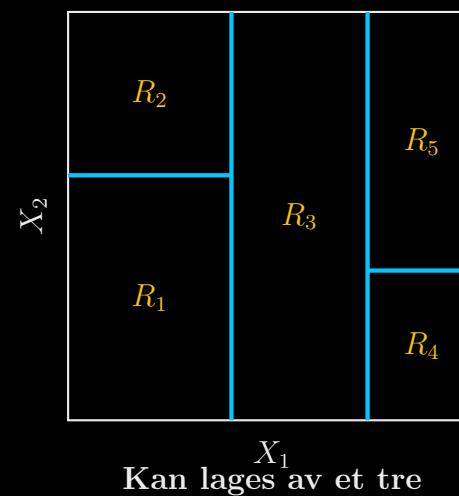
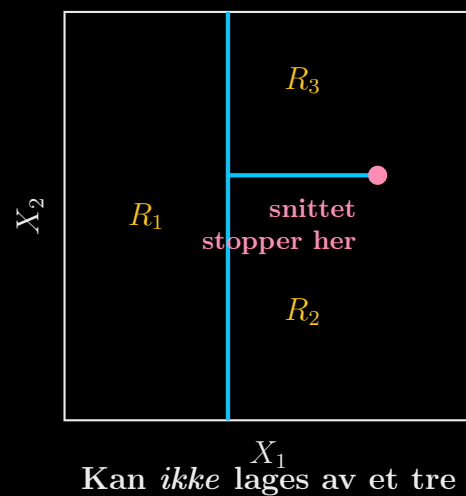
$$\text{RSS} = \sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2,$$

der $\hat{y}_{R_j}$ er gjennomsnittsresponsen i boks j .

Merk: Problemet

Det er **beregningsmessig umulig** å prøve alle mulige oppdelinger av rommet i J bokser. Antallet vokser eksplosivt. Vi trenger en snarvei.

Hvilke oppdelinger kan et tre lage?

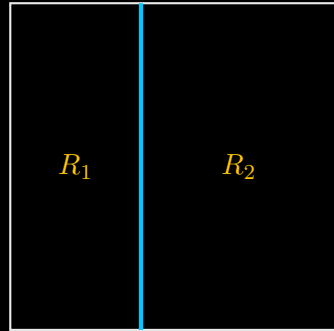


Spørsmål: Se på figuren over

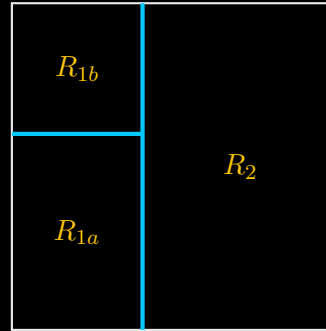
Hvorfor kan ikke venstre oppdeling produseres av et beslutningstre? Hva er det med *rekkefølgen* på splittene som gjør det umulig?

Svar: hver splitt deler en *hel* region i to

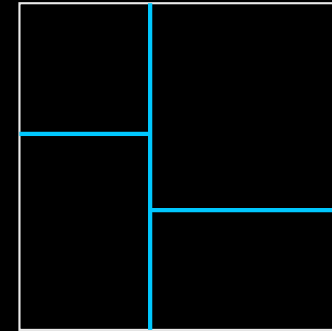
Når vi splitter, kutter vi alltid tvers gjennom en eksisterende region — aldri delvis. Venstre figur krever et snitt som stopper midt inne i rommet uten å dele noe helt. Umulig.



Steg 1: splitt på X_1 ved t_1



Steg 2: splitt *kun* R_1



Steg 3: splitt *kun* R_2

Merk: Konsekvens

Alle regioner et tre kan lage er **akseparallelle rektangler**. Det er både treets styrke (tolkbarhet) og dets svakhet (det kan ikke tegne en skrå linje).

Rekursiv binær splitting

Algoritmen

Top-down: vi starter på toppen, der alle observasjonene er i én region, og deler oss nedover.

Grådig (*greedy*): i hvert steg velger vi den splitten som er best *akkurat nå* — vi ser ikke framover mot hva som blir mulig i neste steg.

For en gitt variabel j og terskel s definerer vi to halvplan

$$R_1(j, s) = \{X \mid X_j < s\} \quad \text{og} \quad R_2(j, s) = \{X \mid X_j \geq s\},$$

og vi leter etter det paret (j, s) som minimierer

$$\sum_{i: x_i \in R_1(j, s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i: x_i \in R_2(j, s)} (y_i - \hat{y}_{R_2})^2.$$

Deretter gjentar vi — men nå splitter vi **én av de to nye regionene**, ikke hele rommet. Så har vi tre regioner. Så fire. Vi fortsetter til et stoppkriterium slår inn, for eksempel: ingen node har færre enn 5 observasjoner.

Diskuter i grupper: Finn den beste splitten for hånd

En liten bedrift har seks ansatte. Vi vil predikere **månedslønn** Y (1000 kr) ut fra **ansiennitet** X (år).

X (år)	1	2	3	6	7	9
Y (1000 kr)	300	320	340	500	520	540

1. Hvor mange kandidat-terskler s finnes? (Hint: midtpunktene mellom naboverdier.)
2. Regn ut RSS for splitten $s = 2.5$. Skriv opp begge gruppegjennomsnittene først.
3. Finn den splitten som gir lavest RSS. Vis regnestykket for minst tre kandidater.
4. Hva er RSS **uten** noen splitt i det hele tatt?
5. **Utvidelse:** hvor mye av variasjonen forklarer den beste splitten alene?

Løsning: RSS for hver kandidat-splitt

s	$\bar{y}_{\text{venstre}}$	$\bar{y}_{\text{høyre}}$	RSS
1.5	300.0	444.0	44 320
2.5	310.0	475.0	25 300
4.5	320.0	520.0	1 600
6.5	365.0	530.0	25 300
8.0	396.0	540.0	44 320
Ingen splitt ($\bar{y} = 420$)			61 600

Kontrollregning for $s = 4.5$:

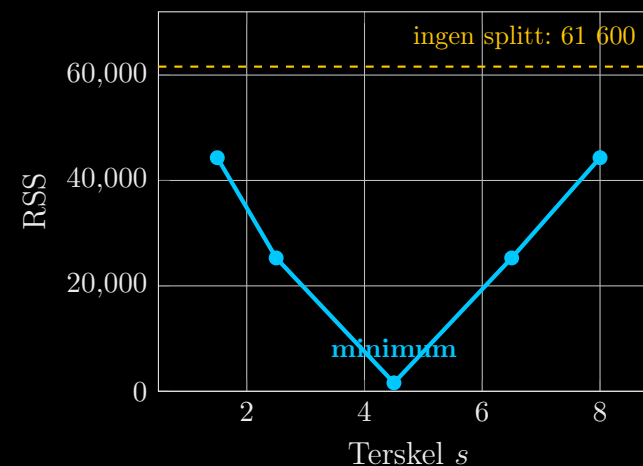
$$\text{venstre} = \{300, 320, 340\}, \bar{y} = 320$$

$$\Rightarrow 400 + 0 + 400 = 800$$

$$\text{høyre} = \{500, 520, 540\}, \bar{y} = 520$$

$$\Rightarrow 400 + 0 + 400 = 800$$

Til sammen 1 600. En eneste splitt fjerner $1 - 1600/61600 = 97,4$ % av RSS.



Merk:

RSS-kurven er ikke glatt — den er definert i endelig mange punkter. Algoritmen prøver rett og slett alle, og velger det laveste. Med p prediktorer og n observasjoner er det maksimalt $p(n - 1)$ kandidater i hver node.

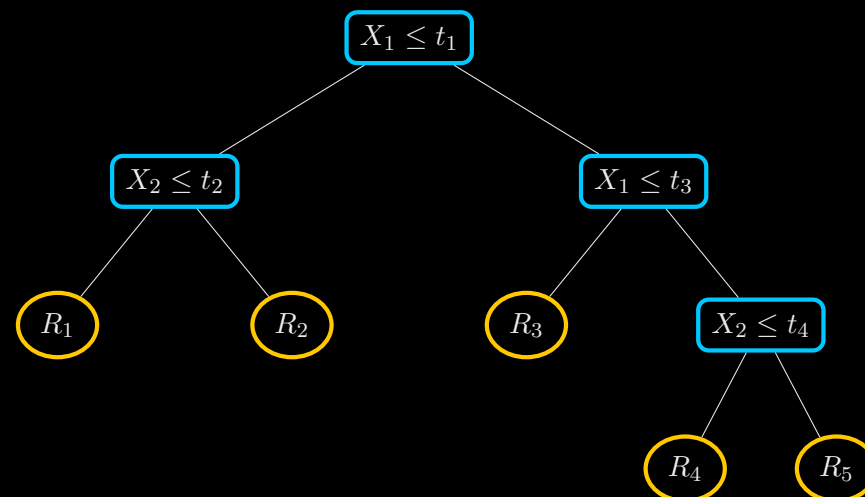
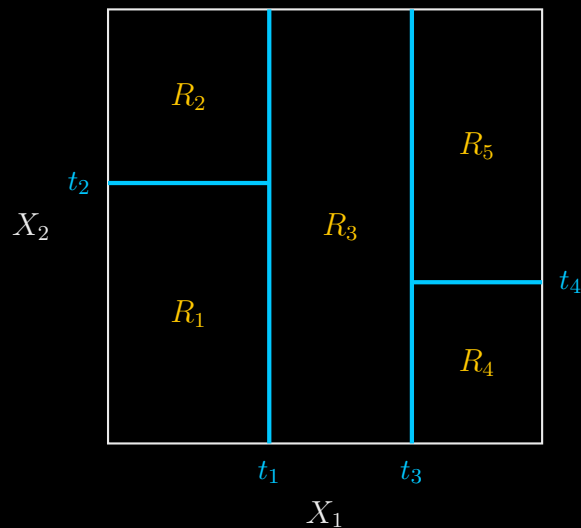
Hva betyr det at algoritmen er “grådig”?

Eksempel: En analogi

Du kjører fra Bardufoss til Tromsø uten kart. I hvert veikryss velger du den avkjøringen som gir mest høydemeter ned akkurat der. Du kommer sannsynligvis fram — men kanskje ikke via den korteste ruten.

- Algoritmen ser **ett steg fram**. Den vurderer ikke om en litt dårligere splitt nå kunne åpnet for to fantastiske splitter senere.
- Vi får derfor ikke garantert det *globalt* beste treet — bare et godt tre, raskt.
- Dette er en bevisst avveining: den optimale løsningen er ikke beregnbar, den grådige er.

Fem regioner: treet og oppdelingen side om side



Spørsmål: Oversett begge veier

Dekk til høyre halvdel og tegn treet ut fra rektanglene. Dekk så til venstre halvdel og tegn rektanglene ut fra treet. Klarer du begge veier, har du forstått modellen.

Problemet: treet vokser inn i himmelen

Hva skjer om vi lar rekursiv binær splitting kjøre helt til hvert blad har én observasjon?

- Treningsfeilen blir **null**. Hver observasjon får sin egen prediksjon.
- Vi har ikke lært en sammenheng — vi har memorert datasettet.
- Bias $\rightarrow 0$, men **variansen eksploderer**. Nøyaktig samme mekanisme som i kapittel 2 og 5.

Merk: En naiv løsning som ikke fungerer

“Stopp å splitte når reduksjonen i RSS blir liten.”

Dette er for kortsiktig. En tilsynelatende verdiløs splitt tidlig i treet kan være det som gjør en *svært* god splitt mulig lenger ned. Fordi algoritmen er grådig, ser den ikke dette.

Strategien som fungerer

Dyrk et **stort** tre T_0 først. Beskjær det **etterpå**.

Kostnadskompleksitets-beskjæring | *Cost-complexity pruning*

Vi vil ikke sjekke alle mulige deltrær — det er alt for mange. I stedet innfører vi en straff for antall blader $|T|$ og lar en parameter $\alpha \geq 0$ styre avveiningen:

$$\sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$$

- $\alpha = 0$: ingen straff. Løsningen er hele det store treet T_0 .
- α stor: hvert blad koster mye, og vi ender med et lite tre.
- Når α øker jevnt, faller greiner av i en **forutsigbar, nøstet rekkefølge**. Vi får dermed en hel sekvens av kandidat-trær nesten gratis.

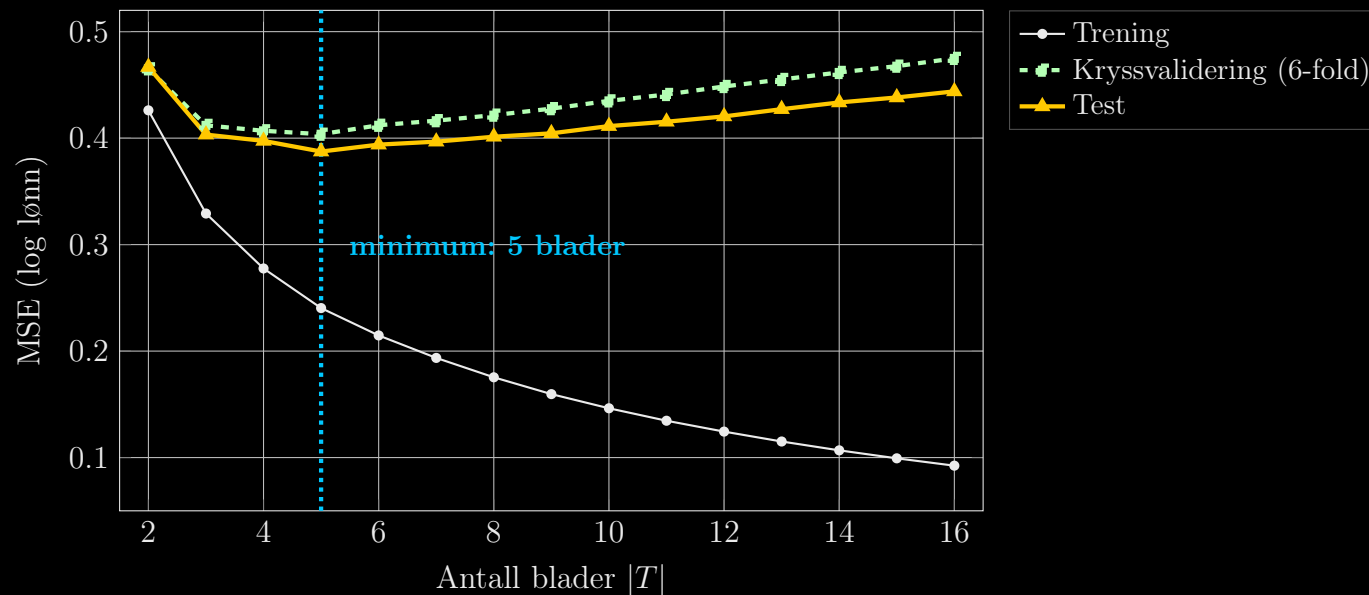
Spørsmål: Grensetilfellene

1. Hva blir treet når $\alpha = 0$? Hvor mange blader har det?
2. Hva blir treet når $\alpha \rightarrow \infty$? Hva predikerer det da — og hvilken modell fra kapittel 3 tilsvarer dette?
3. Hvorfor kan vi **ikke** velge α ved å se på treningsfeilen?
4. Vi kjører 10-folds kryssvalidering og finner optimal α . Hvilket datasett bruker vi til slutt for å bygge det endelige treet?

Merk: Svar på 2 — verdt å dvele ved

Når $\alpha \rightarrow \infty$ sitter vi igjen med ett eneste blad: rotnoden. Treet predikerer $\bar{y}$ for alle. Det er nøyaktig **nullmodellen** $Y = \beta_0 + \varepsilon$. Hele treets kompleksitetsspekter går altså fra “gjennomsnittet” til “memorering”.

Bevis for at beskjæring hjelper: feil mot tresstørrelse



Hitters, $\log(\text{Salary})$ mot 9 prediktorer. Gjennomsnitt over 300 tilfeldige 50/50-delinger, 6-folds kryssvalidering innenfor hvert treningssett.

Les figuren: treningsfeilen (hvit/svart) faller monotont — den vil alltid gjøre det. Test- og CV-feilen har U-form. CV-kurven ligger litt over testkurven, men **minimerer på samme sted**. Det er nettopp derfor vi kan bruke CV til å velge α når vi ikke har et testsett.

Algoritme 8.1: bygg et regresjonstre

1. Bruk rekursiv binær splitting til å dyrke et stort tre på treningsdataene. Stopp først når hvert blad har færre enn et minimum antall observasjoner.
2. Bruk kostnadskompleksitets-beskjæring på det store treet. Dette gir en **sekvens** av beste deltrær, indeksert av α .
3. Velg α med K -folds kryssvalidering. For hver fold $k = 1, \dots, K$:
 - (a) Gjenta steg 1 og 2 på alle foldene unntatt den k -te.
 - (b) Beregn MSE på den utelatte folden, som funksjon av α .Snitt over foldene og velg den α som gir lavest gjennomsnittsfel.
4. Returnér deltreet fra steg 2 som svarer til den valgte α — tilpasset på **hele** treningssettet.

Merk: Pass opp for data-lekkasje

Legg merke til at kryssvalideringen *gjentar hele prosedyren* innenfor hver fold, inkludert selve tredyrkingen. Å dyrke treet én gang på alt og så kryssvalidere beskjæringen ville lekket informasjon fra valideringsfolden.

Klassifiseringstrær: når Y er en kategori

Alt er som før — bortsett fra to ting.

1. **Prediksjonen i et blad** er ikke gjennomsnittet, men den **hyppigst forekommende klassen** blant treningsobservasjonene der. Vi er ofte også interessert i *klasseandelene* i bladet — de forteller hvor sikre vi er.
2. **Vi kan ikke bruke RSS** til å velge splitt. Vi trenger et mål på hvor “ren” en node er.

La $\hat{p}_{mk}$ være andelen av treningsobservasjonene i node m som tilhører klasse k .

Det opplagte første forslaget er **klassifikasjonsfeilraten**:

$$E = 1 - \max_k (\hat{p}_{mk})$$

altså andelen i noden som *ikke* tilhører flertallsklassen.

Merk: Men

Klassifikasjonsfeilraten er ikke følsom nok til å dyrke treet med. Vi skal se hvorfor — med et konkret eksempel — om to sider.

To bedre mål: Gini-indeks og entropi

Gini-indeksen

$$G = \sum_{k=1}^K \hat{p}_{mk} (1 - \hat{p}_{mk})$$

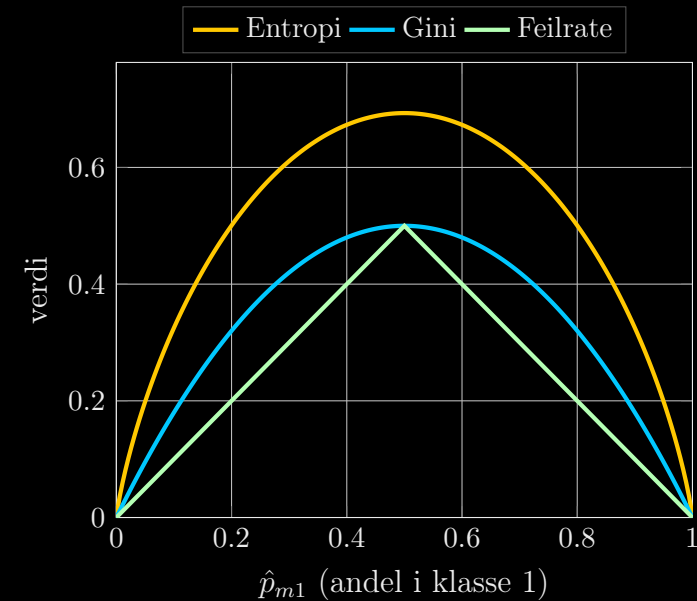
Et mål på total varians på tvers av de K klassene. G blir liten når alle $\hat{p}_{mk}$ ligger nær 0 eller 1 — altså når noden er **ren**.

Entropi (kryss-entropi)

$$D = - \sum_{k=1}^K \hat{p}_{mk} \log \hat{p}_{mk}$$

Siden $0 \leq \hat{p}_{mk} \leq 1$ er hvert ledd ikke-negativt. Også D går mot null når noden er ren.

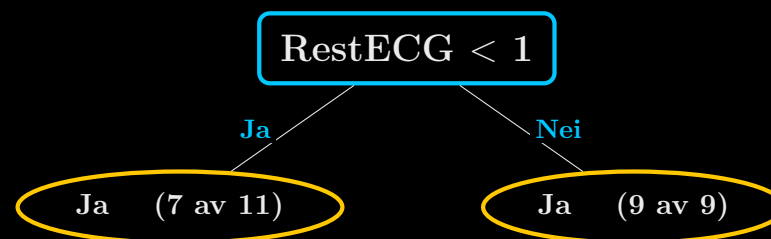
De to er numerisk svært like. I praksis spiller valget sjelden stor rolle.



To klasser, så $\hat{p}_{m2} = 1 - \hat{p}_{m1}$. Alle tre er null i endepunktene og maksimale ved $\hat{p} = 0.5$. Forskjellen: feilraten er **lineær** (spiss), Gini og entropi er **krumme**.

Noderenhet i praksis — et forvirrende, men lærerikt fenomen

I ISLPs Heart-eksempel (303 pasienter, Y = hjertesykdom ja/nei) finnes en splitt der **begge** de resulterende bladene predikerer “Ja”.



Spørsmål: Hvorfor i all verden splitte her?

Prediksjonen blir jo den samme uansett. Splitten reduserer ikke feilraten med ett eneste tilfelle. Hva vinner vi?

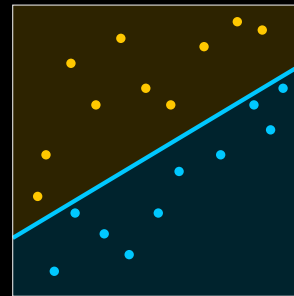
Svar: vi vinner **sikkerhet**. Havner en ny pasient i høyre blad, er alle 9 treningsobservasjonene der “Ja” — vi kan si det med stor trygghet. Havner hun i venstre blad, er svaret fortsatt “Ja”, men bare $7/11 = 64\%$ av observasjonene støtter det.

Splitten forbedrer altså Gini og entropi selv om den ikke forbedrer feilraten. **Det er et poeng for en analytiker, ikke bare for en algoritme:** et svar med sannsynlighet 1.00 og et svar med sannsynlighet 0.64 er ikke det samme svaret.

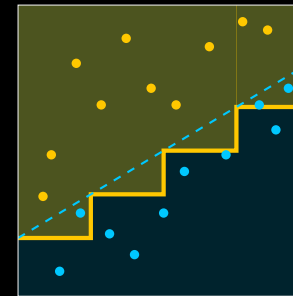
Trær versus lineære modeller: hvilken vinner?

Lineær regresjon antar $f(X) = \beta_0 + \sum_{j=1}^p X_j \beta_j$, mens et tre antar $f(X) = \sum_{m=1}^M c_m \cdot \mathbf{1}(X \in R_m)$.

Sann grense:
lineær

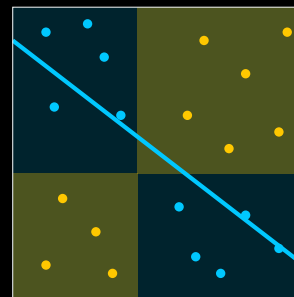


Lineær modell

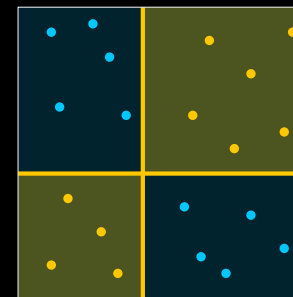


Beslutningstre

Sann grense:
ikke-lineær



Lineær modell



Beslutningstre

Øverst: treet må trappe seg fram langs en skrå grense — lineær regresjon vinner.
treffer perfekt.

Nederst: den lineære modellen har ingen sjanse — treet

Diskuter i grupper: Når ville dere valgt et tre i en økonomisk analyse?

1. Dere skal estimere **priselastisiteten** for et produkt og rapportere den til ledelsen. Tre eller regresjon? Begrunn.
2. Dere skal predikere hvilke kunder som kommer til å **si opp abonnementet** neste måned, slik at kundeservice kan ringe dem. Tre eller logistisk regresjon?
3. Et forsikringsselskap må kunne **forklare for en kunde** hvorfor premien ble som den ble. Hvordan påvirker det metodevalget?
4. Deres modell skal predikere BNP-vekst for et år med **høyere rente enn noe år i datasettet**. Hva skjer med treets prediksjon? (Tenk på hva bladene inneholder.)

Merk: Hint til spørsmål 4

Et tre predikerer alltid gjennomsnittet i et blad. Bladene er bygget av data vi **har sett**. Hva betyr det for ekstrapolasjon? Sammenlign med hva $\hat{\beta}_1 \cdot x$ gjør for en x -verdi langt utenfor datasettet — og vurder hvilken av de to feilene som er farligst.

Fordeler og ulemper med trær

Fordeler

- + Trær er svært enkle å forklare — ofte enklere enn lineær regresjon.
- + De speiler menneskelig beslutningstaking bedre enn de fleste andre metoder.
- + De kan vises grafisk og tolkes av ikke-eksperter (hvis de er små).
- + De håndterer kategoriske prediktorer naturlig, uten dummyvariabler.
- + Interaksjoner og ikke-lineariteter oppstår automatisk.
- + Ingen skalering nødvendig, og manglende verdier kan håndteres elegant.

Ulemper

- Prediksjonskraften er typisk **lavere** enn for de øvrige metodene i kurset.
- Trær er **ikke robuste**: en liten endring i dataene kan gi et helt annet tre.
- De kan ikke ekstrapolere utenfor treningsdataenes verdiområde.
- Trinnvis konstante prediksjoner er en dårlig modell for glatte sammenhenger.

Ustabilitet er ikke en teoretisk bekymring — se selv

Vi deler **Hitters** tilfeldig i to halvdelar, dyrker et tre med 3 blader på den ene halvdel, og gjentar med fire ulike tilfeldige delinger. Samme data, samme metode, samme innstillinger:

Deling	Første splitt	Andre splitt
1	Years < 4.5	AtBat < 400
2	Years < 4.5	Hits < 117.5
3	Years < 3.5	RBI < 43
4	Years < 4.5	Hits < 118

- Den **andre** variabelen bytter mellom AtBat, Hits og RBI.
- Selv terskelen i den *første* splitten flytter seg fra 4.5 til 3.5.
- Prediksjonene er ikke like ustabile som strukturen — men fortellingen vi ville skrevet i f.eks. en rapport, er en helt annen.

Merk: Dette er hovedmotivasjonen for neste forelesning

Høy varians er et problem vi *kan* løse. Løsningen er ikke å lage ett bedre tre, men å lage **mange** og snitte dem.

Python steg 1: data og et lite, lesbart tre

```
import numpy as np
import sklearn.model_selection as skm
from sklearn.tree import (DecisionTreeRegressor as DTR, plot_tree, export_text)
from ISLP import load_data
from ISLP.models import ModelSpec as MS

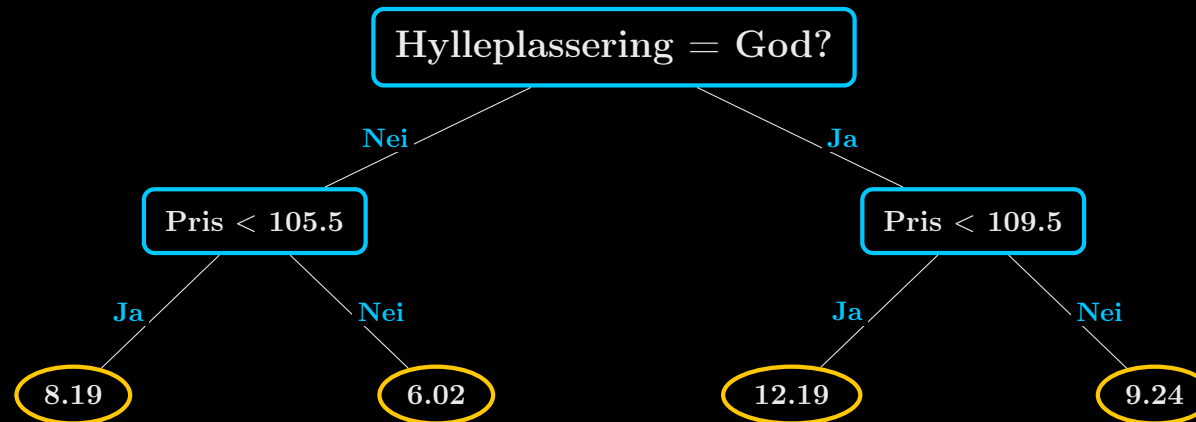
Carseats = load_data('Carseats') # 400 butikker, salg av barneseter
design = MS(Carseats.columns.drop('Sales'), intercept=False)
D = design.fit_transform(Carseats) # one-hot-koder ShelfLoc, Urban, US
navn = list(D.columns)
X, y = np.asarray(D), Carseats['Sales']

tre = DTR(max_leaf_nodes=4, random_state=0) # tvinger fram et lite tre
tre.fit(X, y)
print(export_text(tre, feature_names=navn, decimals=2))
```

```
|--- ShelfLoc[Good] <= 0.50
| |--- Price <= 105.50 --> Sales = 8.19 (n = 108)
| |--- Price > 105.50 --> Sales = 6.02 (n = 207)
|--- ShelfLoc[Good] > 0.50
| |--- Price <= 109.50 --> Sales = 12.19 (n = 28)
| |--- Price > 109.50 --> Sales = 9.24 (n = 57)
```

`plot_tree(tre, feature_names=navn)` tegner samme tre grafisk.

Hva sier treet om barneseter?



Spørsmål: Regn på tallene

1. Hvor mye faller salget når prisen krysser terskelen, gitt **dårlig/middels hylleplass**? Og gitt **god hylleplass**?
2. Er disse to tallene like? Hva betyr det for en lineær modell $\text{Sales} = \beta_0 + \beta_1 \text{Pris} + \beta_2 \text{GodHylle} + \varepsilon$?
3. Hva ville du sagt til butikksjefen som spør: "Skal jeg bruke penger på bedre hylleplass, eller sette ned prisen?"

Fasit på 1: $8.19 - 6.02 = 2.17$ ved dårlig/middels hylle, mot $12.19 - 9.24 = 2.95$ ved god hylle. Preiseffekten er **større** når produktet står godt plassert. Treet fant denne interaksjonen uten at vi ba om den.

Python steg 2: dyrk stort, beskjær med kryssvalidering

```
Xtr, Xte, ytr, yte = skm.train_test_split(X, y, test_size=0.3, random_state=1)

# 1) Dyrk et stort tre uten begrensninger
stort = DTR(random_state=0).fit(Xtr, ytr)

# 2) Hent sekvensen av alfa-verdier fra kostnadskompleksitet
sti = stort.cost_complexity_pruning_path(Xtr, ytr)

# 3) Velg alfa med 10-folds kryssvalidering
kfold = skm.KFold(10, shuffle=True, random_state=1)
grid = skm.GridSearchCV(stort,
                        {'ccp_alpha': sti.ccp_alphas},
                        refit=True, cv=kfold,
                        scoring='neg_mean_squared_error')
grid.fit(Xtr, ytr)

# 4) Det beskjaerte treet er tilpasset paa nytt paa hele treningssettet
beste = grid.best_estimator_
print(beste.tree_.n_leaves, np.mean((yte - beste.predict(Xte))**2))
```

	Antall blader	Trenings-MSE	Test-MSE
Ubeskåret tre	279	0.00	5.92
Beskåret tre ($\alpha = 0.26$)	6	2.65	5.09

Trenings-MSE på nøyaktig **null** med 279 blader: treet har memorert alle 280 treningsobservasjonene. Beskjæringen kaster 273 blader og gjør testfeilen **bedre**. RMSE = 2.26, altså typisk bom på ca. 2 260 solgte enheter.

Det beskjærte treet — og hvilke variabler som betyr noe

```
|--- ShelveLoc[Good] <= 0.50
| |--- Price <= 105.50
| | |--- Age <= 50.50 --> 10.07
| | |--- Age > 50.50 --> 7.32
| |--- Price > 105.50
| | |--- ShelveLoc[Medium] <= 0.50 --> 4.80
| | |--- ShelveLoc[Medium] > 0.50 --> 6.63
|--- ShelveLoc[Good] > 0.50
| |--- Price <= 135.00 --> 10.88
| |--- Price > 135.00 --> 6.82
```

```
beste.feature_importances_
```

```
ShelveLoc[Good] 0.449
Price 0.347
Age 0.112
ShelveLoc[Medium] 0.093
Advertising 0.000
```

Merk: Les feature_importances_ med varsomhet

Tallet er den totale reduksjonen i RSS som splitter på den variabelen gir, normalisert til å summere til 1. Det er **ikke** en koeffisient, ikke en elastisitet, og ikke en kausal effekt. En variabel kan få 0.000 fordi den er irrelevant — eller fordi den er sterkt korrelert med en variabel som ble valgt først.

Python steg 3: klassifiseringstre

```
from sklearn.tree import DecisionTreeClassifier as DTC
from sklearn.metrics import accuracy_score
from ISLP import confusion_table

High = np.where(Carseats.Sales > 8, "Yes", "No") # hoyt salg, ja/nei
Xtr, Xte, htr, hte = skm.train_test_split(X, High, test_size=0.3, random_state=1)

clf = DTC(criterion='entropy', random_state=0).fit(Xtr, htr)
# criterion='gini' er standard; 'entropy' gir kryss-entropi

sti = clf.cost_complexity_pruning_path(Xtr, htr)
grid = skm.GridSearchCV(clf, {'ccp_alpha': sti.ccp_alphas},
                        refit=True, cv=kfold, scoring='accuracy').fit(Xtr, htr)
beste = grid.best_estimator_
print(confusion_table(beste.predict(Xte), hte))
```

Truth	No	Yes
Predicted		
No	57	15
Yes	13	35

Nøyaktighet = $(57 + 35)/120 = 76,7 \%$.

Ubeskåret tre: 41 blader, **100 %** riktig på treningsdata, 76,7 % på test.

pruned: 34 blader, 76,7 % på test.

Fallgruver du kommer til å møte

1. **scikit-learn** beskjærer ikke av seg selv. Standardinnstillingene dyrker treet til hvert blad er rent. Du *må* sette `ccp_alpha`, `max_depth` eller `max_leaf_nodes` — ellers har du memorert dataene.
2. **Splittregelen er $\leq$, ikke $<$.** `export_text` skriver `Price <= 105.50`. ISLP-boka skriver `Price < 105.5`. Samme splitt, ulik notasjon — terskelen legges mellom to nabo-verdier.
3. **Kategoriske variabler må one-hot-kodes.** `MS(...)` gjør dette for deg. Sender du en tekstkolonne rett inn, får du en feilmelding.
4. **`random_state` betyr noe.** Uavgjort mellom to like gode splitter brytes tilfeldig. Uten `random_state` får du ulikt tre hver gang du kjører koden.
5. **Treet vurderes på testsettet, ikke på treningssettet.** En trenings-MSE på 0.00 er sjelden et godt tegn.
6. **Ekstrapolasjon finnes ikke.** Sender du inn en pris høyere enn noen i treningsdataene, får du bare gjennomsnittet i det ytterste bladet.

Oppsummering av kapittel 8.1

- Et tre deler prediktorrommet i **akseparallelle rektangler** og predikerer gjennomsnittet (regresjon) eller flertallsklassen (klassifisering) i hvert rektangel.
- Regionene finnes med **rekursiv binær splitting**: top-down og grådig. Kriteriet er RSS for regresjon, Gini eller entropi for klassifisering.
- Klassifikasjonsfeilraten er for lite følsom til å dyrke treet med, men brukbar til å beskjære det.
- Et fullvokst tre har lav bias og **svært høy varians**. Vi dyrker stort og **beskjærer etterpå** med kostnadskompleksitet, der α velges ved kryssvalidering.
- Trær er suverene på tolkbarhet og håndterer interaksjoner og kategorier naturlig — men de er **ustabile** og typisk svakere på prediksjon enn kursets øvrige metoder.

Merk: Til eksamen bør du kunne

Lese et tre, tegne den tilsvarende partisjonen, regne RSS for en kandidat-splitt, regne Gini og entropi for en node, og forklare hvorfor beskjæring er nødvendig. Bruk trær i prosjektet, da vil dere få håndfast eksempelbruk av trær!

Fra én ekspert til en folkemengde

Vi har sett at ett enkelt tre er svakt og ustabilt. Men hva om vi kombinerer mange?

Analogien:

- Ville du stolt blindt på rådet fra **én** ekspert — som kanskje har en dårlig dag?
- Eller ville du stolt mer på **gjennomsnittet av 500 eksperter** som har sett på problemet fra ulike vinkler?

Eksempel: Neste forelesning: kapittel 8.2

- **Bagging:** bootstrap mange datasett, dyrk ett tre per datasett, snitt. Reduserer varians.
- **Random forests:** som bagging, men hver splitt får bare se $m \approx \sqrt{p}$ tilfeldige variabler. Dekorrelerer trærne.
- **Boosting:** dyrk trærne sekvensielt, der hvert nytt tre lærer av restene fra de forrige.

Prisen vi betaler: vi mister det tolkbare treet. Vi bytter tolkbarhet mot treffsikkerhet.